Bringing General-Purpose Parallel Programming to Near-Data Compute

Harry Kim

Chief Product Officer

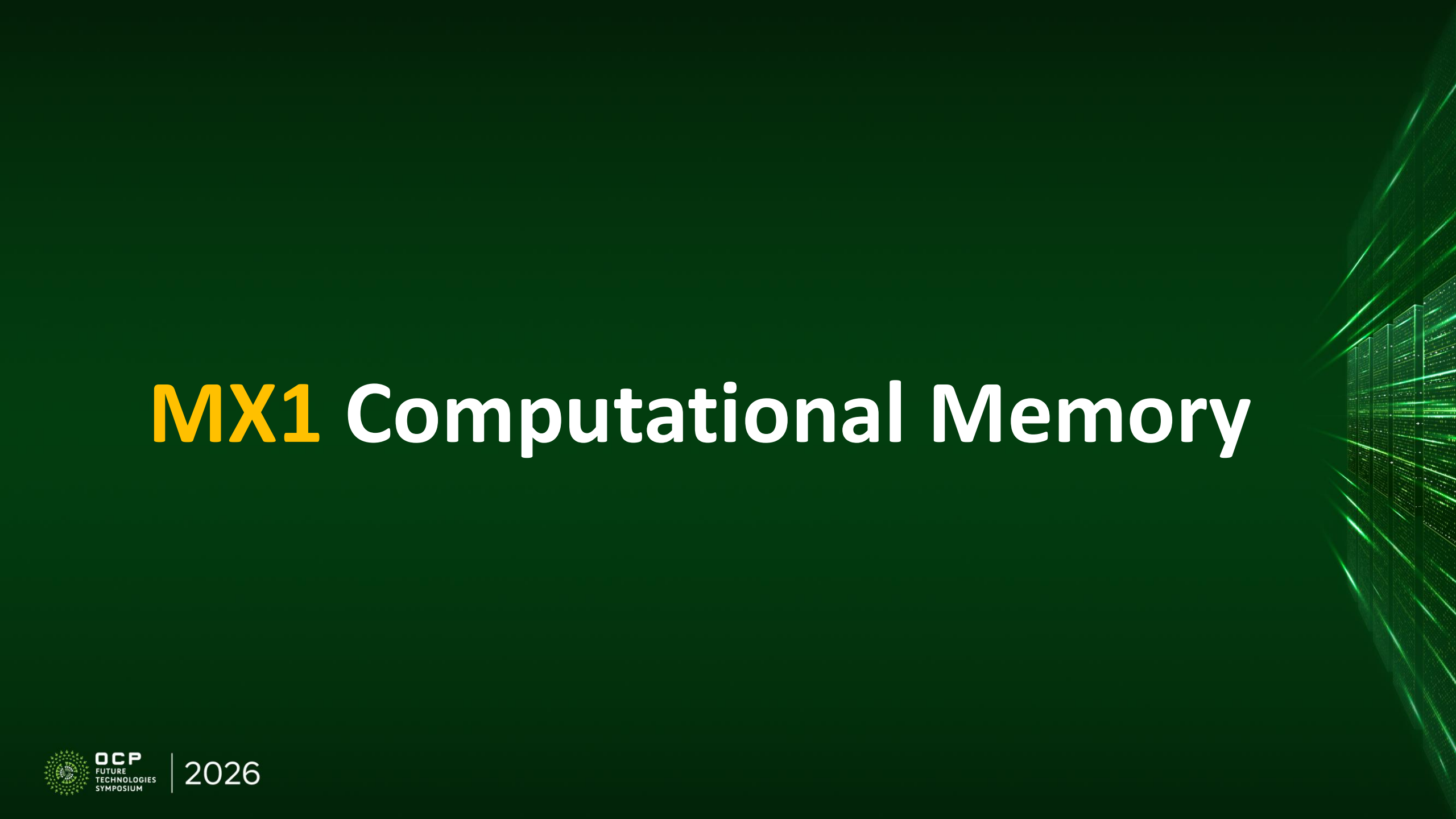
XCENA



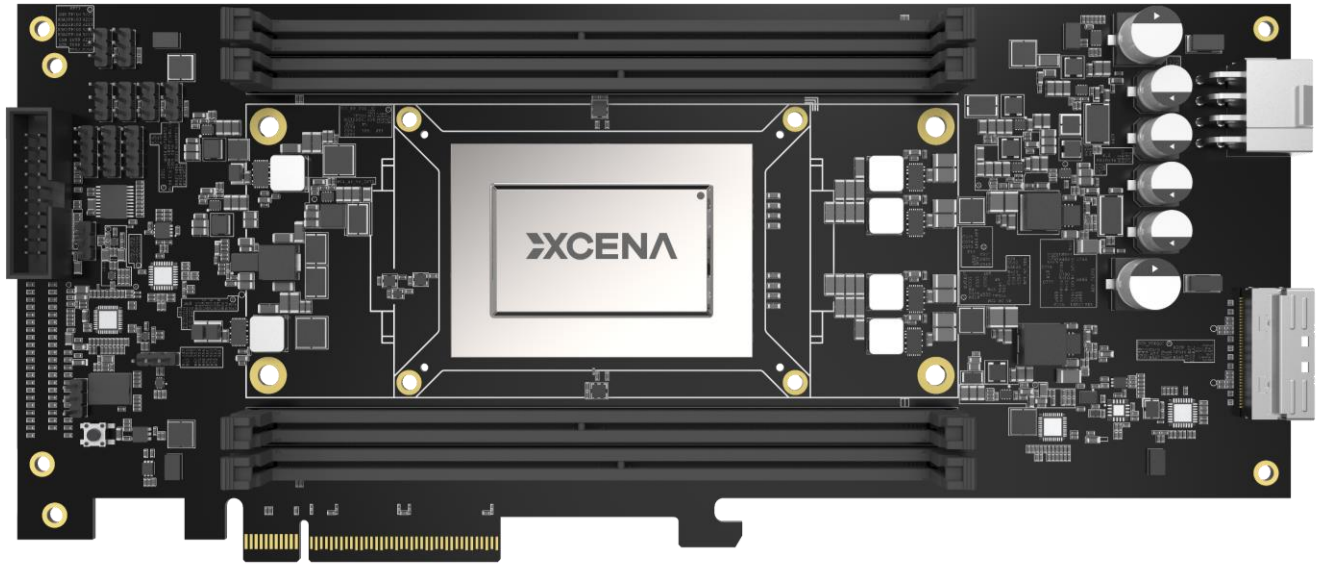
OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

MX1 Computational Memory



MX1P at a Glance



CXL Memory Expansion

- 4ch DDR5 8400MT/s, up to 2TB
- CXL 3.2 Type 3, PCIe Gen6
- Chip-kill RAS

Infinite Memory Extension

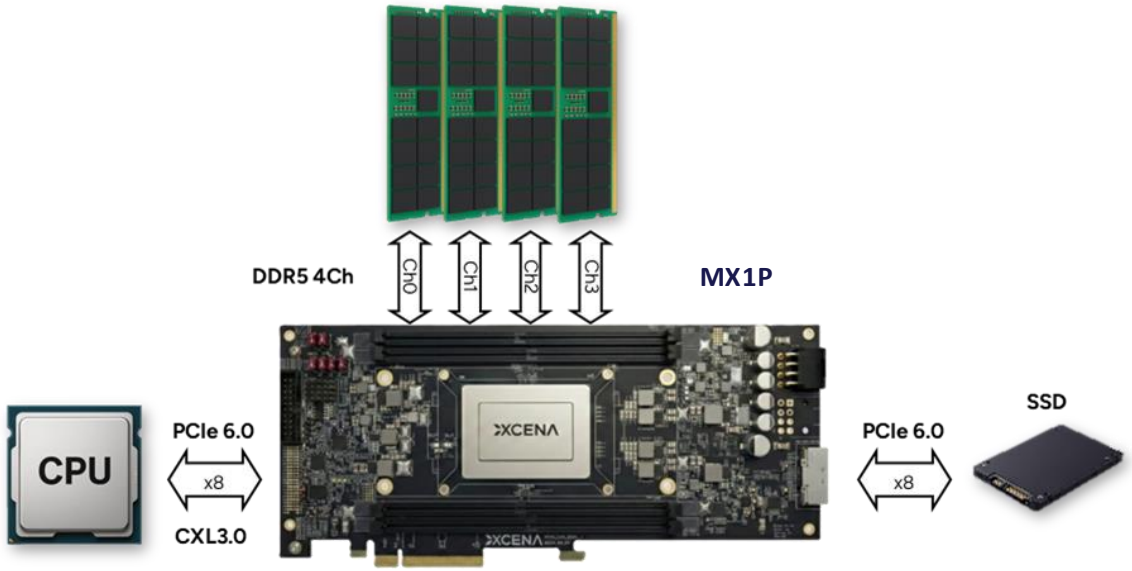
- SSD on MX1 root port = CXL memory
- Host sees full SSD capacity
- DRAM as cache tier

Near-Memory Compute

- 3,072 RISC-V MU cores, 24 subsystems
- FP32 / FP16 vector engine
- User kernels via PXL — xcena.com/SDK

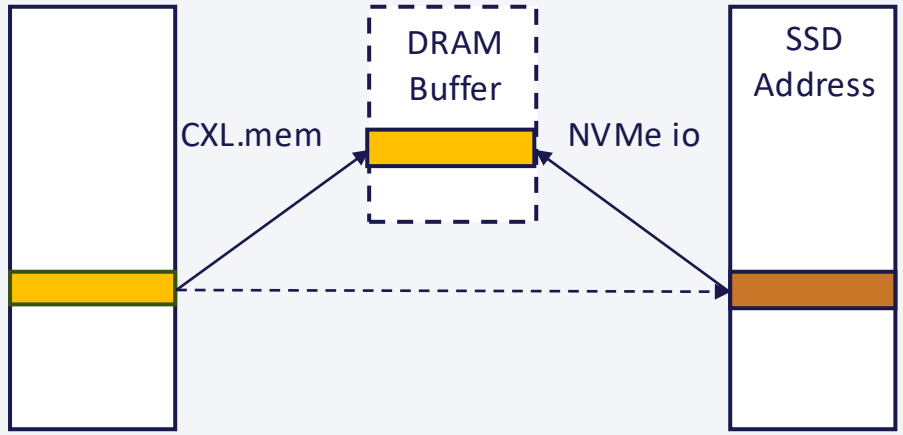
CXL Memory Expansion and InfiniteMemory®

MX1 presents a CXL endpoint to the host and acts as a PCIe root complex to the SSDs, DDR5 is the on-device cache; NAND is the capacity layer.



- MX1P – PoC, PCIe Gen6 x8 (CXL host), x8 (PCIe)
- MX1 – Production, PCIe Gen6 2x8 (CXL host), 2x4 (PCIe)

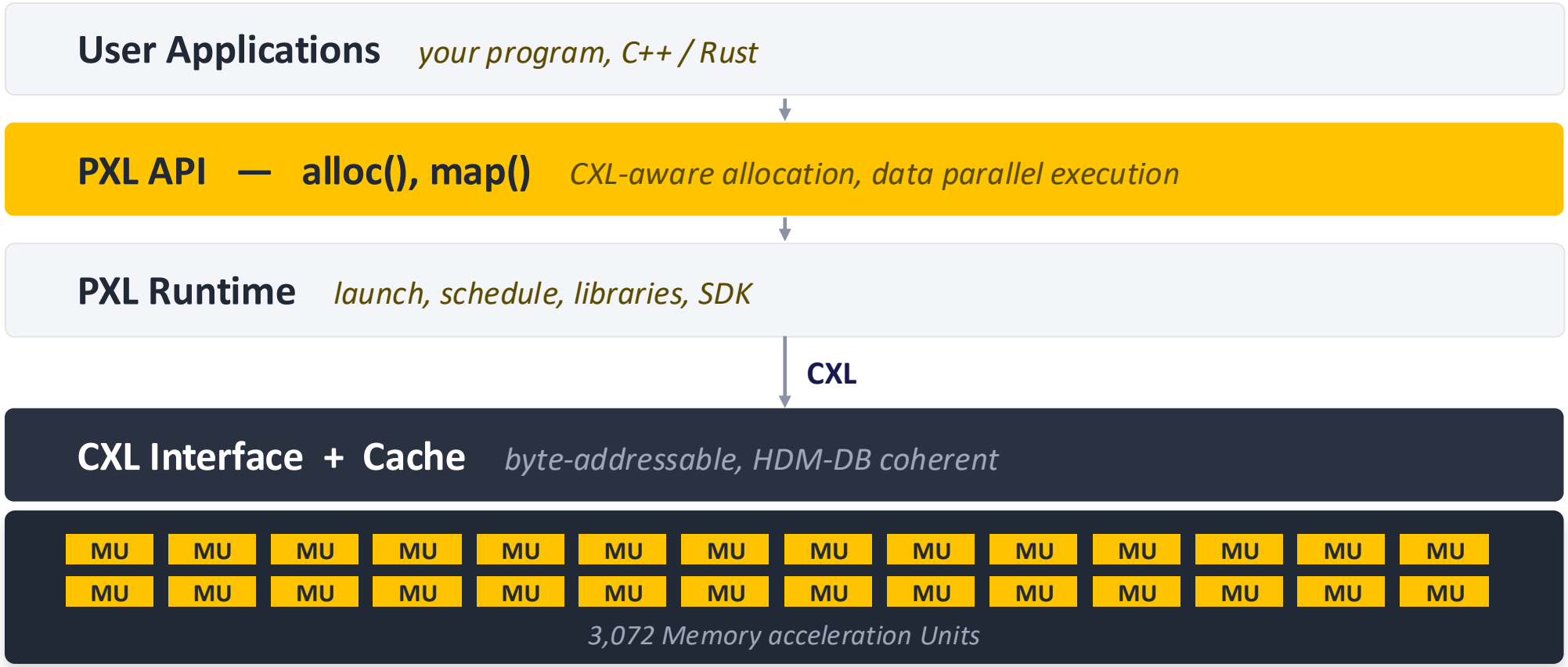
Memory address translation



- The CPU transparently accesses SSD-backed memory through CXL.mem.
- MX1 translates the CXL memory space into SSD address space; the DRAM buffer hides SSD latency.

MX1 Architecture and Software Layers

From C++/Rust applications down to CXL memory, the MX1 stack runs kernels on the memory-side cores, keeping data in place with no host round trips.



Memory Model



Allocate, Don’t Copy

Placement is the design decision, data movement is the expensive operation.

Copy-based offload

- Host copies the working set to the device
- Device computes, then copies results back
- Every call pays I/F bandwidth and latency
- I/F bandwidth is limited and latency is long

Allocate in place (MX1P)

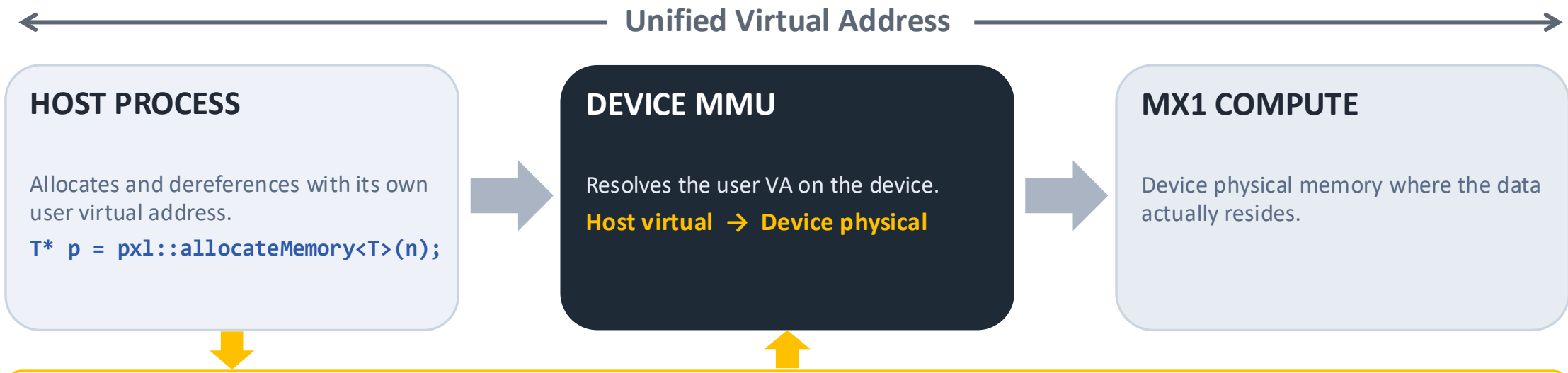
- Allocate the working set directly in CXL memory
- Cores compute where the data already lives
- No round trips, no duplicate buffers
- Placement, decided once, drives performance

Don’t copy data. Move computation.

On MX1P you decide data placement once, and the memory-side cores run right next to it.

One Address Space, Host and Device

User virtual addresses everywhere; the device MMU resolves them locally.



Same pointer on both sides.

Pointer chasing and fine-grained host/device offload become natural.

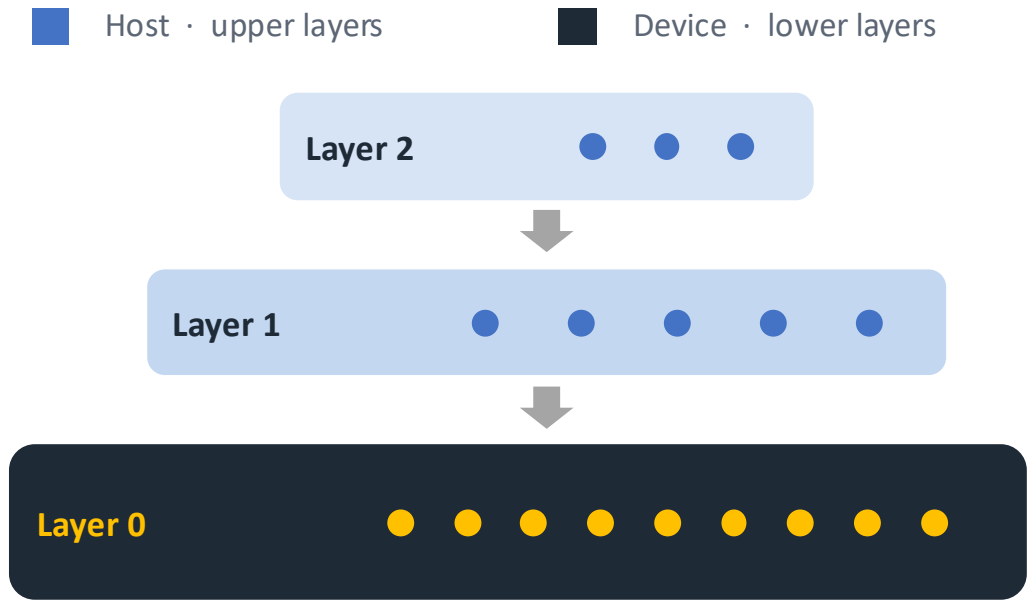
Pointer chasing across host and device is natural

Split the work at the pointer level, with the option to rebalance at runtime.

Enables fine-grained offloading

- Shared virtual address: pointers resolve on host or device
- Chase pointers on the device, with no nodes copied back
- Move the host/device boundary per query
- Bigger index? push more layers down to the device
- Decide the balance dynamically, at runtime

e.g. HNSW search descends layer by layer



Memory Model in Code

Allocate once in CXL memory, share the pointer, compute in place.

<host.cpp>

```cpp
// Open a context on the target device
pxl::Context* ctx = pxl::createContext(deviceId);

// Load the compiled kernel binary (.mubin) onto the device
pxl::Module*  mod = pxl::createModule("xvec_kernel.mubin");

// Allocate device (CXL) memory: query/result buffers
auto* in  = (HnswSearchInF32*) ctx->memAlloc(…);
auto* out = (HnswSearchOutF32*)ctx->memAlloc(…);
auto [queries, results] = allocBuffers(ctx, numQueries, dimension, topK);

// Load the HNSW index (base vectors + graph) into CXL memory
loadIndexToCxlMemory(in, …);

// Build the job and map the kernel over tasks
pxl::Job* job = ctx->createJob(numSub);
job->load(mod);
pxl::Function* func = mod->createFunction("xvecKernelHnswSearchL2");
pxl::Map* map = job->buildMap(func, taskCount);

// Launch the kernel and wait for completion
map->execute(in, out, …);
map->synchronize();
```

<device.cpp>

```cpp
void xvecKernelHnswSearchL2(
    HnswSearchInF32*  in, HnswSearchOutF32* out, …)
{
    const HnswGraph& graph = in->graph;

    // L2 distance over base vectors, streamed into an internal buffer.
    L2Distance dist(…);

    // Each task handles its slice of the query batch.
    for (const HnswQuery& query : partitionQueries(in->batchedQueries))
    {
        dist.setQuery(query); // query stays in the internal buffer
        auto entry  = greedyDescent(graph, dist); // upper layers
        auto result = beamSearchLevel0(graph, dist, entry,
query.efSearch);
        storeTopK(out, query, result, in->topK);
    }
}
```

Placement chosen at allocation

One address same user VA on host and device

Offload map->execute() runs the kernel on the MU cores.

Coherency at the Cache-Line Level

CXL 3.x Back-Invalidation lets the device keep host caches coherent.

HDM-H

Host-managed region

- The host manages coherence for this region
- Device flushes its caches to publish updates
- Best for data that rarely changes
- Raw vectors and embeddings live here

HDM-DB

Device-managed, with Back-Invalidation

- Device issues CXL 3.x Back-Invalidation
- Host caches stay coherent automatically
- Best for frequently updated data
- Index and hash structures live here

Place stable data in **HDM-H**, mutable data in **HDM-DB**, to balance cache-flush cost against Back-Invalidation traffic.

Programming Model



Thread-Pool Execution, Not SIMT

128 threads per subsystem run the same program with independent state.

One program, many threads

128 threads per Subsystem execute the same code.

Independent, not lockstep

Each thread has its own state — divergent branches don't serialize. It's a thread pool, not a warp.

Latency hidden by parallelism

Many independent accesses stay in flight, so CXL's long latency and limited bandwidth are absorbed.

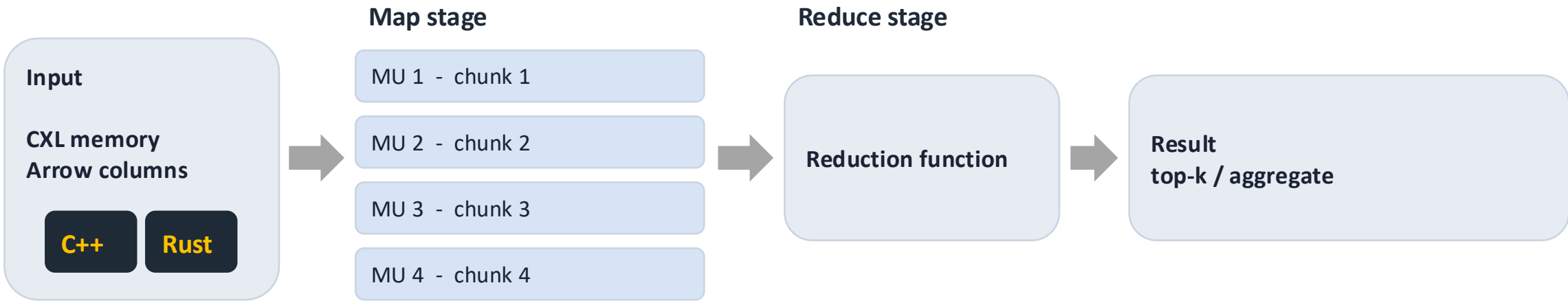


A Subsystem is the unit of Job allocation, and one Job can be assigned multiple Subsystems

Jobs can belong to different Tenants

Map-Reduce Over the Cores

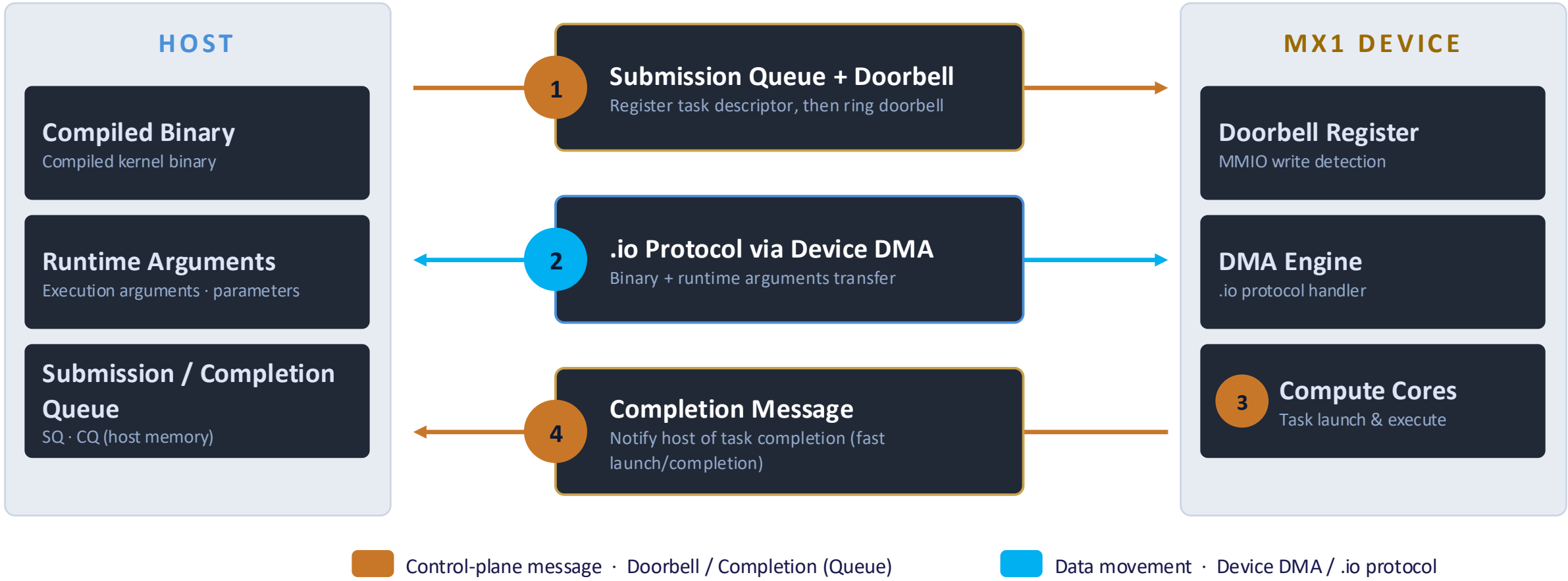
A familiar abstraction for distributed parallel processing across many cores.



map splits the working set, the runtime schedules chunks across cores, **reduce** combines the partials.

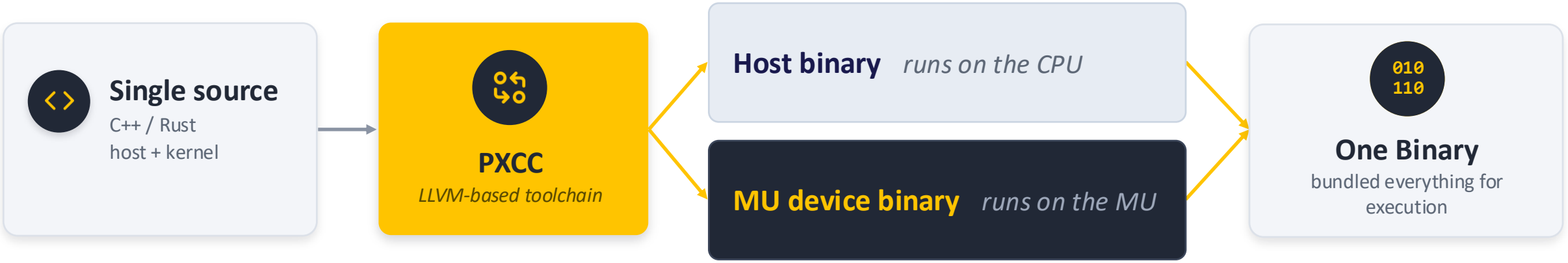
Control Plane

Binary and args reach MX1 via Device DMA (.io); Queue/Doorbell/Message drive fast task launch and completion.



Heterogeneous Compiler

PXCC builds host and device binaries from a single source file, keeping the code readable and easy to maintain.



PXCC targets the host CPU and the memory-side MU cores from single source code.

PXCC example

Simplifies kernel invocation and improves code maintainability.

https://xcena-dev.github.io/sdk_release/tools/pxcc.html#pxcc--heterogeneous-cc-compiler

Controllability vs. Abstraction

<host.cpp>

```cpp
// Open a context on the target device
pxl::Context* ctx = pxl::createContext(deviceId);

// Load the compiled kernel binary (.mubin) onto the device
pxl::Module*  mod = pxl::createModule("xvec_kernel.mubin");

// Allocate device (CXL) memory: query/result buffers
auto* in  = (HnswSearchInF32*) ctx->memAlloc(…);
auto* out = (HnswSearchOutF32*)ctx->memAlloc(…);
auto [queries, results] = allocBuffers(ctx, numQueries, dimension, topK);

// Load the HNSW index (base vectors + graph) into CXL memory
loadIndexToCxlMemory(in, …);

// Build the job and map the kernel over tasks
pxl::Job* job = ctx->createJob(numSub);
job->load(mod);
pxl::Function* func = mod->createFunction("xvecKernelHnswSearchL2");
pxl::Map* map = job->buildMap(func, taskCount);

// Launch the kernel and wait for completion
map->execute(in, out, …);
map->synchronize();
```

```cpp
#include "mu/mu.hpp"
#include "pxl/pxl.hpp"

// Device kernel — extracted and compiled for the device by pxcc.
__pxl_kernel__ void sort_with_ptr(int* data, int size)
{
    int idx = mu::getTaskIdx();
    std::sort(data + idx * size, data + idx * size + size);
}

int main()
{
    const int testCount = 2048, sortSize = 64;
    auto* data = pxl::allocateMemory<int>(0, testCount * sortSize);
    for (int i = 0; i < testCount; i++)
        for (int j = 0; j < sortSize; j++)
            data[i * sortSize + j] = sortSize - j;

    auto result = pxl::Launcher().execute<sort_with_ptr>(testCount, data, sortSize).run();
    if (result.status != pxl::Result::Success)
    {
        printf("FAIL: %s\n", result.errorMessage.c_str());
        pxl::releaseMemory(data);
        return 1;
    }

    // Verify every sub-array is sorted.
    bool sorted = verify(…);
    printf("%s\n", sorted ? "PASS" : "FAIL");

    pxl::releaseMemory(data);
    return sorted ? 0 : 1;
}
```

```
> pxcc++ sort.cpp -o sort   # compile host + device and link in one step
> sudo ./sort
```

The XCENA SDK

Runtime, compiler, debugger and profiler, plus a QEMU emulator, all in one Docker image at xcena.com/SDK.

Runtime

- PXL library
- Resource and memory management

Compiler (PXCC)

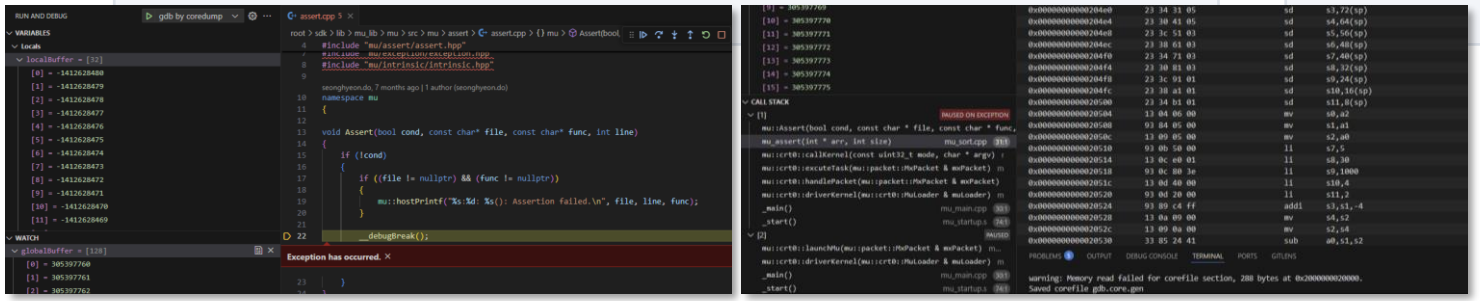
- C++ and Rust front-end
- LLVM-based; host and device targets

Debugger

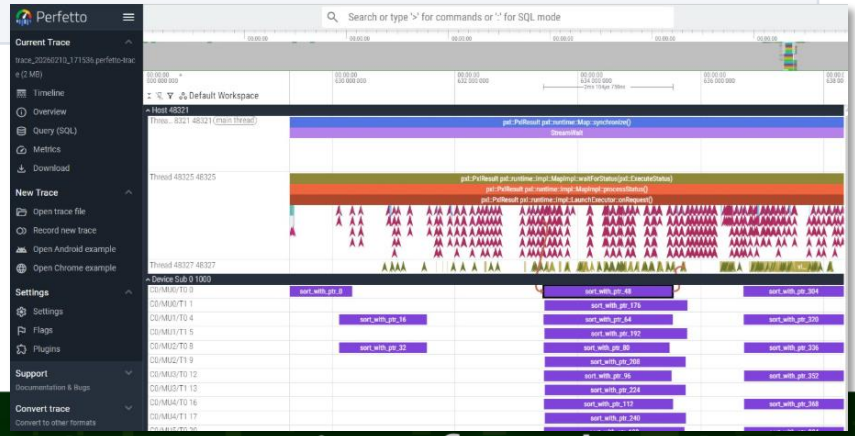
- Step through host and device kernels
- Inspect state across the cores

Profiler

- Bottlenecks across host and cores
- Guides placement and offload



gdb + vscode



Applications



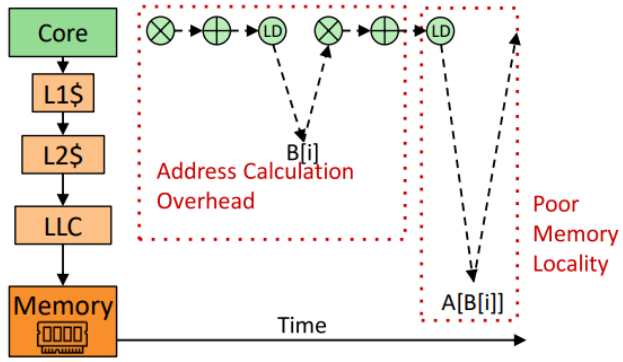
Sparse / Indirect Accesses

Many opportunities for maximizing memory "goodput"

Address calculation and *indirect memory accesses* can take significant time.

- Direct multidimensional memory accesses: Parthenon's *cell* and *flux* variables
- Indirect sparse data access: AMG2023's *SpMV* or *Gauss Seidel* kernels
- Indirect pointer chasing: Branson's and Sparta's *move* and *sort* particle kernels

Benchmark	Access Pattern	Instruction Count ⊗	⊕	LD
Parthenon	A[i][j][k][l][m][n]	6	6	1
AMG2023	A[B[i]]	2	2	2
Branson	A[B[i].idx + const]	2	4	2
	A[B[i] + const * i]	3	3	2
Sparta	C[B[A[i].idx].idx]	3	5	3
	C[A[i].idx][B[A[i].idx]]	6	8	5



source: https://www.kisacoresearch.com/sites/default/files/presentations/galen_shipman_los_alamos_labs_0.pdf

"Let memory do what it does best"

Scatter/gather wastes bandwidth

- Writing 8 bytes pulls in an entire 64-byte cache line
- Large strides defeat the cache prefetcher
- Over CXL, this amplification costs link bandwidth
- Scatter/gather is the worst access pattern for CXL

CXL approach: scatter/gather near the device

1. Expand capacity and bandwidth with CXL
2. Device gathers (in parallel) → host reads sequentially
   Host writes sequentially → device scatters (in parallel)
3. Run compute on the device

XVector index support, accelerated on MX1P

Vector Index support

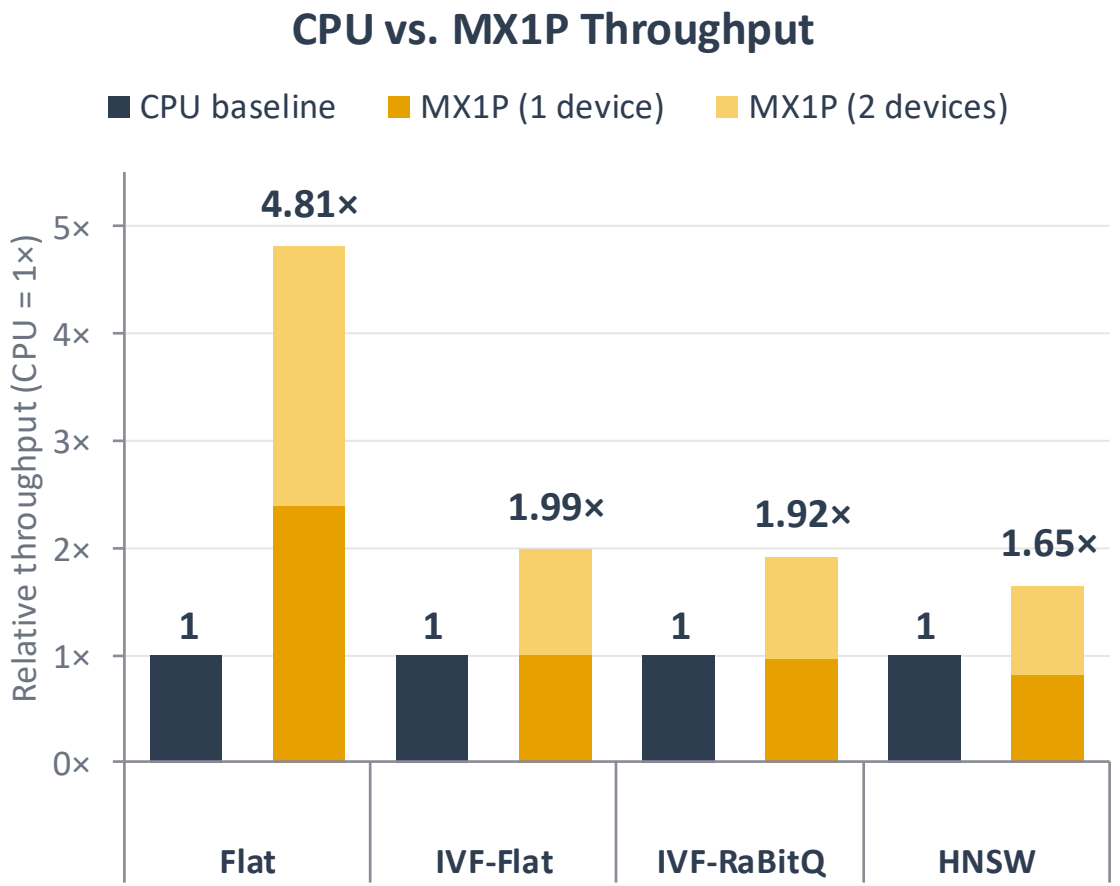
- **Flat**
- **IVF-Flat**
- **RaBitQ**
- **HNSW**

FAISS integration

Integrates with the FAISS API and stays portable to other vector engines.

Scales linearly with MX1P

Each MX1P adds bandwidth and near-memory compute together, so throughput grows in step.

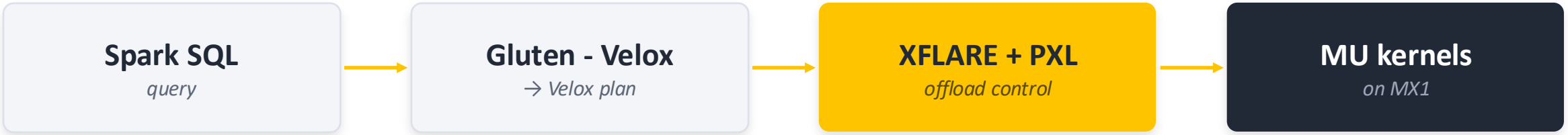


CPU baseline

Intel GNR 6767P, sub-NUMA 32core, 4ch

XFLARE

A query engine that offloads analytics to MX1P. *(in development)*



Operators run as MU kernels

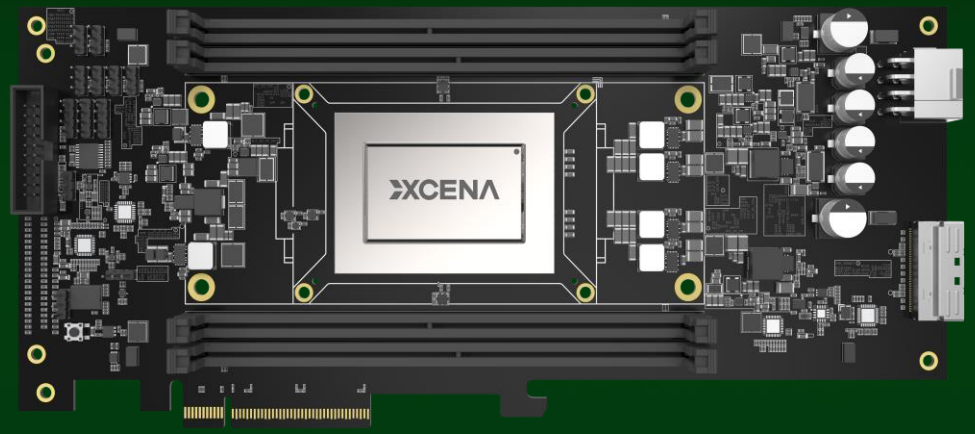
- TableScan
- Decompress
- Join/OrderBy
- Aggregate

Data stays in Memory

- Arrow columns live in CXL DRAM
- Substrait + Arrow keep it portable
- intermediate data isn't shipped to the CPU

Thank You!

“Let memory do what it does best”



MX1 CXL Computational Memory

XCENA
Xcelerate Your Intelligence

Booth #A2

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

